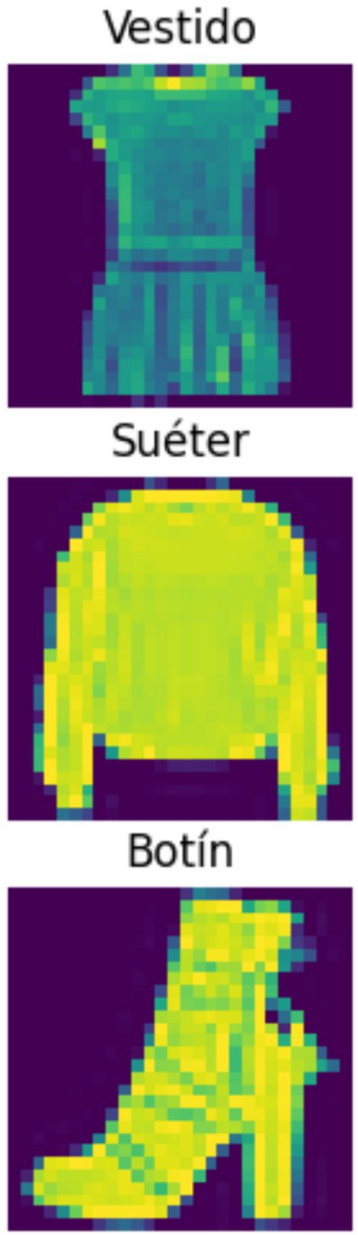
Clasificación de imágenes

Visión computarizada

Aprendizaje supervisado:

Se utilizan datos de entrenamiento etiquetados



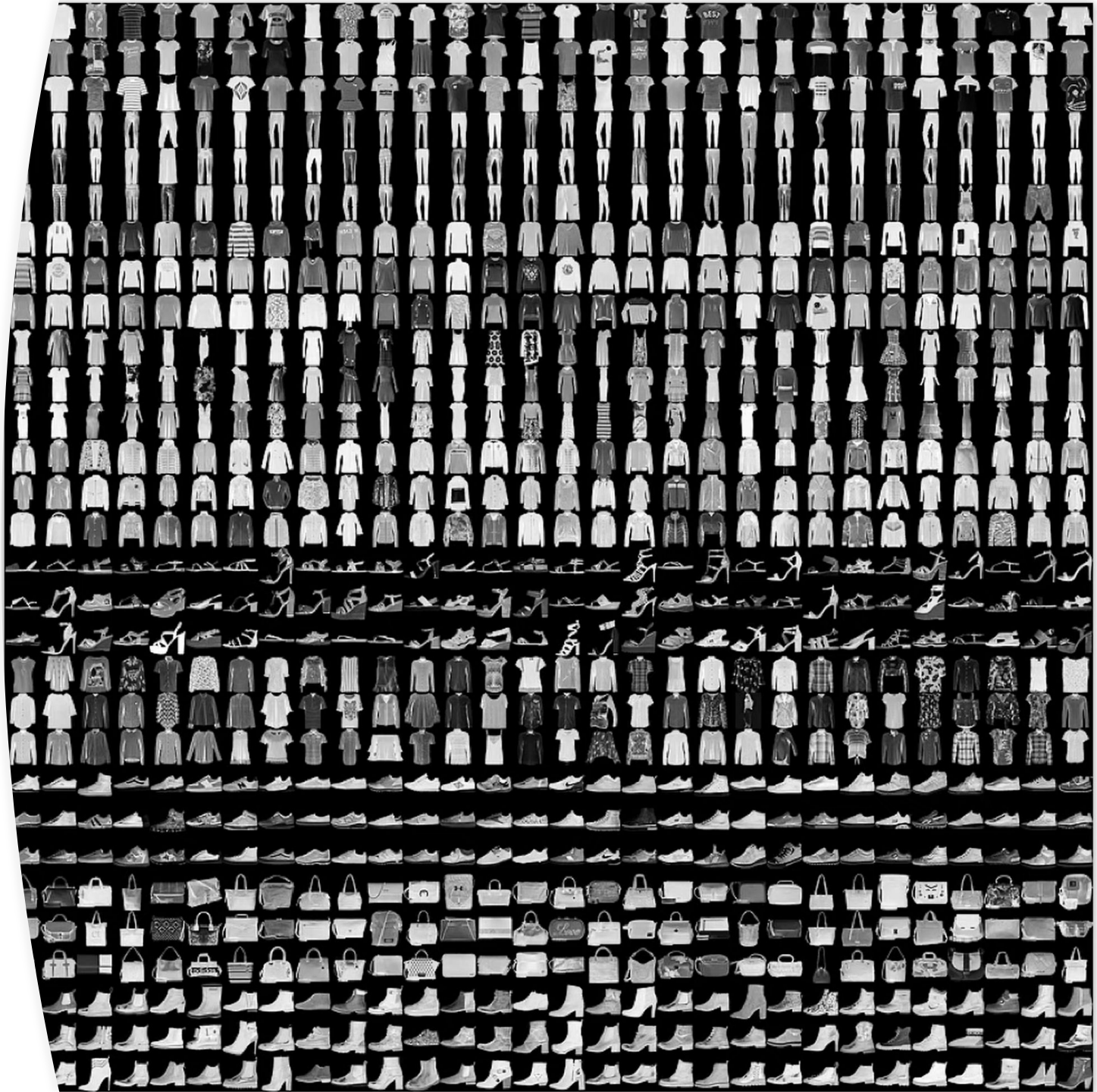
MNIST dataset

- Utilizaremos el conjunto de datos MNIST
- El NIST produjo la base de datos Fashion MNIST, es una gran base de datos de imágenes de ropa utilizados para entrenar sistemas de procesamiento de imágenes.
- MNIST esta incluido en Tensorflow



Fashion MNIST

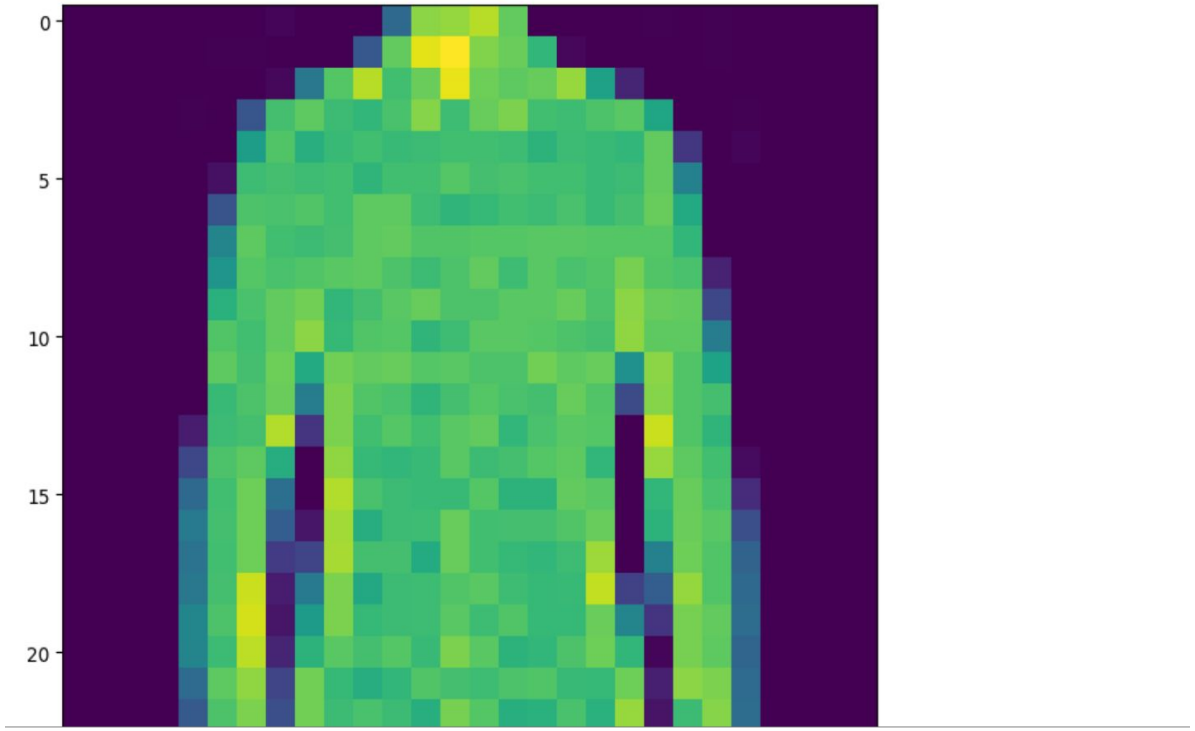
- Es una BD que posee 70,000 imágenes (60,000 entrenamiento y 10,000 test)
- Existen 10 categorías
- Es decir que existen 7,000 ejemplos de cada categoría incluidos zapatos.
- Imágenes de 28x28

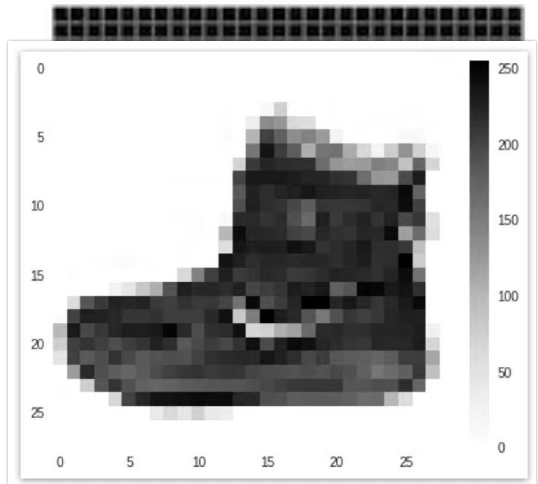


Formato de cada imagen

- Las imágenes de MNIST son imágenes de 28x28 pixeles
- Imágenes pequeñas serán procesadas por una computadora (neurona artificial) mas rápido
- Cada pixel puede ser representado en valores desde cero hasta 250 (1 byte por pixel).

```
[[  0   0   0   0   0   0   0   3   0   0   0  87 211 215 228 194   0   0   0   0   1   0   1   0   0   0   0   0]
 [  0   0   0   0   0   0   1   1   0   0   0  70 194 245 255 207 197 169   5   0   0   0   0   1   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   5 102 188 228 178 197 247 199 192 196 216 146  26   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   1   0  68 180 192 175 169 182 209 176 196 205 178 176 184 190 150   0   0   1   0   0   0   0]
 [  0   0   0   0   0   0   0 142 186 161 173 178 173 175 178 178 176 165 175 173 169 194  41   0   3   0   0   0   0]
 [  0   0   0   0   0   0  11 176 180 176 178 167 178 178 188 180 182 178 178 173 175 194 112   0   0   0   0   0   0]
 [  0   0   0   0   0   0  66 184 182 186 178 192 192 176 167 171 178 175 182 173 180 196 156   0   0   0   0   0   0]
 [  0   0   0   0   0   0 116 192 178 175 180 192 194 186 186 188 188 190 190 188 188 188 169   0   0   0   0   0   0]
 [  0   0   0   0   0   0 133 188 182 186 190 192 184 175 184 194 176 190 182 186 203 186 182  24   0   0   0   0   0]
 [  0   0   0   0   0   0 163 182 194 201 169 180 190 196 184 184 190 190 196 184 211 196 194  55   0   0   0   0   0]
 [  0   0   0   0   0   0 186 178 194 211 171 186 188 167 176 190 190 188 184 180 213 192 192 108   0   0   0   0   0]
 [  0   0   0   0   0   0 192 180 199 156 201 194 196 188 190 184 184 201 192 196 129 211 186 148   0   0   0   0   0]
 [  0   0   0   0   0   0 173 184 197 108 205 186 182 167 180 188 182 180 196 186  58 209 186 180   0   0   0   0   0]
 [  0   0   0   0  20 175 180 226  38 205 178 188 178 190 194 169 182 190 188   0 235 186 167   0   0   0   0   0   0]
 [  0   0   0   0  55 184 192 159   0 213 171 169 173 190 175 180 188 192 169   0 216 192 178   7   0   0   0   0   0]
 [  0   0   0   0  87 178 199  93   0 226 182 175 173 173 188 165 165 194 190   0 169 196 182  32   0   0   0   0   0]
 [  0   0   0   0 104 180 199  76  15 220 159 175 176 196 178 180 180 186 196   0 165 197 190  62   0   0   0   0   0]
 [  0   0   0   0  97 178 199  45  53 222 180 180 157 196 178 171 169 176 218   0 112 199 186  81   0   0   0   0   0]
 [  0   0   0   0 102 180 235  19 106 205 154 175 176 186 190 176 171 173 232  49  76 215 186  87   0   0   0   0   0]
 [  0   0   0   0 117 184 239  15 140 205 171 175 176 190 176 186 171 171 197 116  39 203 194  91   0   0   0   0   0]
 [  0   0   0   0 116 175 230  26 165 190 180 188 173 205 190 165 169 184 199 169   3 203 192  83   0   0   0   0   0]
 [  0   0   0   0  89 192 213  53 201 171 159 169 186 180 176 184 186 173 171 199  22 211 205  89   0   0   0   0   0]
 [  0   0   0   0  72 184 205  62 201 175 182 175 161 203 188 161 163 182 163 216  13 175 209  91   0   0   0   0   0]
 [  0   0   0   0  72 192 165  70 215 161 156 176 184 209 182 178 178 176 167 226  19 148 211 110   0   0   0   0   0]
 [  0   0   0   0  64 205 159  76 201 171 163 163 163 194 171 173 173 171 161 186  55 148 222 129   0   0   0   0   0]
 [  0   0   0   0  66 180 156  47 203 184 190 190 186 220 216 176 190 196 192 220  24 108 178 116   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0 184 230 216 213 203 169 197 190 194 201 194 112   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0  55 119 137 148 127 144 156 142  62   0   0   0   0   0   0   0   0   0   0   0]]
```





pixel 1
pixel 2
pixel 3
pixel 4
pixel 5
pixel 6
pixel 7
pixel 8
pixel 9
pixel 10
pixel 11
pixel 12
pixel 13
pixel 14
pixel 15
pixel 16
pixel 17
pixel 18
pixel 19
pixel 20
pixel 784

Entrenamiento

- Se utilizara la función de perdida **parse_categorical_crossentropy** para clasificacion de imagenes y la funcion de activacion: **Softmax**

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),   # Convierte 28x28 → 784 neuronas
    keras.layers.Dense(128, activation='relu'),   # Capa oculta con ReLU
    keras.layers.Dense(10, activation='softmax')  # Capa de salida (10 clases)
])
```